

```

fn fib[x: int]: int {
  if x < 2 {
    return x;
  } else {
    return fib[x - 1] + fib[x - 2];
  }
}

fn main[a: int, b: float]: void {
  while (a + 2) < int(-b * 3.0) {
    a = fib[a];
  }
}

```

Figure 3: Example program in our programming language.

q and k denote the number of lookbehind symbols and lookahead symbols respectively. For this study, we have opted to implement an $LLP(1, 1)$ parser, in which case the stack configuration changes corresponding to a pair of lookbehind and lookahead symbols can be stored in a pre-computed two-dimensional table. In this case, the grammar must allow constructs to be deduced from two consecutive input symbols. This puts restrictions on the grammar which in turn leads to a limitation in the languages that can be parsed. One could modify the grammar to conform to these restrictions, but the result is a language with inconvenient and unorthodox syntax. Alternatively, one can modify the grammar to accept a super set of the desired syntax and verify full correctness during the semantic analysis stage that follows the parsing stage.

For the purpose of this initial feasibility study, we have designed a custom imperative and procedural programming language with a static type system, bearing resemblance to existing programming languages. As such, the operations performed in our parallel compiler relate to existing compilers. A super set of our language can be parsed using a $LLP(1, 1)$ parser and full correctness is verified in during semantic analysis. An example program is shown in Figure 3. As can be seen, we have had to make a number of compromises such as always enforcing curly braces around compound statements, using square brackets for function applications, parsing `else` and `elif` parts of `if`-statements as separate statements and verifying the relative order of these later, and limiting the number of supported data types. More work is necessary to strike a balance between parsing programming languages with traditional syntax and the degree of parallelism that can be achieved.

2.3 Semantic Analysis

This stage has three major tasks: (1) validate and correct the parse tree for the fact that a super set of the desired grammar was accepted by the parser, (2) analyze the parse tree to determine whether it fits the semantic rules of the language, (3) transform the parse tree to an AST. All of these tasks are implemented as a series of passes. A pass processes the parse tree and may add, replace or remove nodes. Each pass is designed such that it performs small, node-local operations that can be executed for all nodes of the tree in parallel, or by parallelizing large, tree-wide operations in a clever way.

2.4 Intermediate Code Generation

At this point, the AST is transformed into machine code. For this study, we have selected the RISC-V architecture as target because

of the fixed-width instructions and the regularity of the instruction encoding. Part of the generation of instructions is the selection of free registers. Because this is highly dependent on neighboring instructions, this would severely restrict the degree of parallelism that can be achieved. To counter this, we postpone register allocation and first compile to an intermediate representation which uses virtual registers and that is similar to a static single assignment representation of RISC-V. Section 3.5 explains the allocation of physical registers. We allow each register to be written once and read multiple times, but do not allow a register to be written in between writes. The output register number is computed from the instruction location, already satisfying the requirement that each register can only be written once. The actual generation of the instructions is parallelized by processing all nodes at the same depth in the tree simultaneously.

2.5 Optimization

Optimizations can be performed on the intermediate representation that do not require information about the physical registers. Optimizations modify the instruction buffer and parallelization is done at the level of instructions instead of nodes. Many of the feasible optimizations at this stage involve the removal of instructions (e.g., dead code removal or constant folding). However, repeatedly removing instructions from an array-based instruction buffer is an expensive operation, as remaining instructions need to be scattered to new locations and function locations and jump targets need to be updated. Therefore, we employ an instruction mask indicating whether an instruction in the program is enabled, which later stages can use to determine whether an instruction will be optimized away. The actual removal of instructions and correction of data structures is delayed until the final step of the machine code generation.

2.6 Machine Code Generation

At this stage, the intermediate representation is transformed into the final RISC-V machine code. First, *register allocation* will map the infinite set of virtual registers onto the finite set of physical registers. If insufficient registers are available, registers will be flagged for spilling and the necessary stack space will be allocated. In traditional compilers a graph coloring algorithm is typically used to perform register allocation. While the graph coloring algorithm itself can be run efficiently on a parallel architecture like a GPU [5], the fact that each iteration may insert spill instructions would impose a significant overhead due to the array reallocation that is required. Therefore, we opt to implement a greedy algorithm to allocate registers. Although this does not result in mappings as optimal as those produced by a graph coloring approach, a reasonable mapping is still produced for most applications given the large amount of registers present on the RISC-V target architecture.

Additionally, the actual insertion of spill instructions is postponed until register allocation has completed. By doing so, all spill instructions can be inserted in one go *and* this can be combined with the removal of instructions according to the instruction mask. This results in the final instruction buffer. After post-processing the jump instructions and computing the final function offset table, the compilation process has finished and the results can be transferred from GPU to CPU memory.